# Nested Loops

Tong Yi

## Contents

**1 Goals** **1**

**2 Structure of the Project** **1**

**3 Task A: Draw a Diamond Shape of Asterisks** **2**

**4 Task B: Find out All Right Triangles with Integer Sides** **2**

**5 Task C: Coin Combinations** **3**

**6 Task D: Count Frequency of First Name** **4**

6.1 Requirements . . . . . 4
6.2 Sample Input / Output . . . . . 4
6.3 Read a TSV file and Save the First Name in an Array . . . . . 5
6.4 Sort an Array . . . . . 5
6.4.1 Selection Sort . . . . . 5
6.4.2 Bubble Sort . . . . . 5
6.5 Count Frequency of First Names in a Sorted Array . . . . . 6
6.6 Applications of This Task . . . . . 6

## 1 Goals

In this project, we will learn how to work on nested loops.

## 2 Structure of the Project

1. Create directory `nested_loops` to hold codes of the project **if** you have not done so. Said differently, you only need to run the following command once.

```
mkdir nested_loops
```

2. Move to the above directory.

```
cd nested_loops
```

# 3 Task A: Draw a Diamond Shape of Asterisks

1. Create a source file named **diamond.cpp**.
2. Prompt the user with a message like: "Enter the size of a diamond as a positive odd integer: ".

   Read a value into an integer variable `size`, which represents the maximum number of asterisks in any single row.
3. Implement an `input validation loop` that repeats until the user provides a positive odd integer.
4. Print a diamond shape made of asterisks (*) corresponding to the input size.

Here is a sample input/output.

```
Enter the size of a diamond as a positive odd integer: -2
size needs to be a positive odd integer: -7
size needs to be a positive odd integer: 2
size needs to be a positive odd integer: 3
 *
***
 *
```

Submit `diamond.cpp` to gradescope. Note that the grading script generated random int numbers to test. As a result, your output will be different in each running.

# 4 Task B: Find out All Right Triangles with Integer Sides

**Objective:** Write a C++ program (rightTriangle.cpp) that finds all unique right triangles with integer sides up to a user-defined maximum hypotenuse.

1. **Input:** Prompt the user to enter the maximum value for the hypotenuse.
2. **Logic:**
   (a) Find all integer sets (a, b, hypotenuse) such that $a^2 + b^2 = \text{hypotenuse}^2$.
   (b) Constraints: To avoid redundant results (like 3,4,5 and 4,3,5), enforce: $a < b$. Since `hypotenuse` is the longest side in a right triangle, we have

$$1 \leq a < b < \text{hypotenuse}$$

3. **Output Format:**
   (a) Display the results in a numbered list.
   (b) Each line must follow the format:
   Order: a b hypotenuse
   (c) The Order must start at 1 and be `immediately` followed by a colon (:).
4. Here is a sample input/output when the maximum hypotenuse is 25.

```
Enter the maximum hypotenus: 25

   1:    3   4   5
   2:    5  12  13
   3:    6   8  10
   4:    7  24  25
   5:    8  15  17
   6:    9  12  15
   7:   12  16  20
   8:   15  20  25
```

# 5 Task C: Coin Combinations

In this task, complete the following:

1. **File Naming:** Name your source code file `coinCombinations.cpp`.
2. **User Input:** Prompt the user to enter:
   - an amount in dollars (as a decimal, e.g., `0.25`), and
   - the maximum number of cents (pennies) allowed in any combination.
3. **Processing:** Determine all possible combinations of quarters (25¢), dimes (10¢), nickels (5¢), and cents (1¢) that sum to the specified amount. Each valid combination must satisfy the constraint that the number of cents does not exceed the user-defined maximum.
4. **Output:** Display all valid combinations in a neatly formatted table. Each row should include:
   - a sequential label starting from 1, followed by a colon (`:`), and
   - the number of quarters, dimes, nickels, and cents (in that order), separated by spaces.

   If no valid combination exists, output `no solution`.
5. **Sample Input/Output:**

```
Enter amount: 0.25
Enter maximum number of cents: 6
        quarters   dimes nickels   cents
     1:        0       0       4       5
     2:        0       0       5       0
     3:        0       1       2       5
     4:        0       1       3       0
     5:        0       2       0       5
     6:        0       2       1       0
     7:        1       0       0       0
```

**Alternative Case (No Solution):**

```
Enter amount: 0.31
Enter maximum number of cents: 0
        quarters   dimes nickels   cents
no solution
```

# 6 Task D: Count Frequency of First Name

While Tasks A–C used only for loops, nested loops can incorporate different types. In this task, you will use a while loop within a nested structure to sort data and count frequencies (the number of times each item occurs).

## 6.1 Requirements

1. Name the source code `firstNameFrequency.cpp`.
2. Read a TSV (Tab-Separated Values) file containing a header line followed by names in "first name last name" format. Store the first names in a string array. You may assume the file has no more than 100 names, so set your array capacity to 100.
3. Sort the array in alphabetical order.
4. Count the frequency of each first name and output the results.

## 6.2 Sample Input / Output

Given the following `names.tsv` file:

```
firstName lastName
Alice Martinez
An Davis
Jordan Johnson
David Smith
Bob Miller
Alice Williams
An Brown
Charles Garcia
Jordan Chen
Morgan Wang
Alice Rodriguez
```

After reading `names.tsv`, the first names in alphabetic order and their frequency (number of occurrences) are displayed.

```
Enter a file name in .tsv: names.tsv
Alice,3
An,2
Bob,1
Charles,1
David,1
```

```
Jordan,2
Morgan,1
```

## 6.3 Read a TSV file and Save the First Name in an Array

Please refer to Lab 3.

## 6.4 Sort an Array

For the sorting requirement, you may implement either Selection Sort or Bubble Sort.

Note that using the `std::sort` function from the `<algorithm>` library is strictly prohibited; failure to provide your own implementation will result in zero credit.

### 6.4.1 Selection Sort

The Selection Sort algorithm sorts an array by repeatedly finding the minimum element from the unsorted part and putting it at the proper position. The Key Steps are as follows.

1. **Find the Minimum:** Search the unsorted portion of the array to find the index of the smallest element.

   Hint: Initialize your "minimum" as the first element of the current unsorted section to avoid re-checking sorted elements.

2. **Swap:** Swap the smallest element found with the element at the first position of the **unsorted** section.

3. **Repeat:** Move the boundary of the **unsorted** array one index to the right and repeat the above process.

4. **Completion:** For an array with $n$ elements, continue until $n-1$ elements have been processed.

   Note that once $n-1$ elements are in their proper positions, the remaining element (the largest one) is naturally at its proper position.

You may watch a YouTube video on selection sort for a demonstration.

### 6.4.2 Bubble Sort

Two adjacent items are **out-of-order** if they violate the sorting criteria. For an array `arr` of size $n$, where $0 \leq i < n-1$:

- **Ascending:** `arr[i]` and `arr[i+1]` are out-of-order if `arr[i] > arr[i+1]`.
- **Descending:** `arr[i]` and `arr[i+1]` are out-of-order if `arr[i] < arr[i+1]`.

To sort in ascending order, the algorithm proceeds in passes:

1. **Pass 1:** Compare all adjacent pairs from index 0 to $n-1$, swapping out-of-order items. This "bubbles" the largest element to the rightmost position.

2. **Pass 2:** Repeat the process for the remaining $n-1$ items to place the second-largest element.

3. ...

4. **Pass $n-1$:** Compare the final two elements. Once the second-smallest is placed, the smallest is naturally left in the first position.

## 6.5 Count Frequency of First Names in a Sorted Array

Because the array is sorted, identical names are grouped together. The process is as follows:

1. Iterate through the array, incrementing a counter for each occurrence of the same name.
2. Upon reaching a different name (or the end of the array), record the final count for the current name.
3. Reset the counter to be zero and repeat the process for the next unique name.

## 6.6 Applications of This Task

By adding one additional step – not required for this project – to sort the frequencies in descending order, this logic can be extended to identify the most popular first names and analyze naming trends within a population.